Matrice de couverture des fixtures - SCE 2.0 Basic Set v1.8 - pili-pos-android

Distributeur : Bruno Moyse / nuagemagique.dev (Bruxelles, BE) **Produit :** pili-pos-android 1.0.0 (posSwVersion pili-pos-android/1.0.0) **Plateforme :** Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google) **Date :** À effectuer (EV-18 : date de dépôt)

Test pilote : `crates/pos-fiscal/tests/uc_fixtures.rs` **Fixtures source :** `docs/20260821_UC_BasicSet_POS_FDM_1`

Révision du jeu publié : le dossier `docs/20260821_UC_BasicSet_POS_FDM_1.8/` remplace `docs/20250916_UC_BasicSe` Changements de la v1.8 (changelog `CHANGELOG-UC_BasicSet1.8.md`, comparé aux fichiers de la v1.7) : M121, réponse : suppression de l'espace final dans `terminalId` ; tous les horodatages des fixtures portent désormais les millisecondes (`2024-07-29T13:00:00.000+02:00`). Les tests lisent les fixtures de la v1.8 sans autre adaptation.

Chaque dossier UC du Basic Set v1.8 publié par SPF Finances est chargé par le walker de découverte et validé. Le `*_Response.json` (ou `status_response.json`) de chaque fixture est parsé via `serde_json::from_value::<SignResult>` / `FdmStatus` - autrement dit c'est **le même désérialiseur** que le client FDM utilise en production dans `crates/pos-fiscal/src/fdm_client.rs`.

Invariants imposés par UC :

- Les **événements N** (signSale) doivent porter `shortSignature` (40 caractères hex) et `verificationUrl`.
- **Tout événement non-N** ne doit PAS porter `shortSignature` ni `verificationUrl`.
- Les **événements de formation** (label T) doivent omettre `vatCalc` (changelog v1.6 "M171 : correction dans la réponse FDM (pas de vatCalc)").
- Chaque mutation `Variables.json` porte les champs communs SCE 2.0 (`posId`, `posFiscalTicketNo`, `posDateTime`, `deviceId`, `vatNo`, `estNo`, `posSwVersion`, `bookingPeriodId`, `bookingDate`, `ticketMedium`, `employeeId`).

Matrice de couverture

Deux colonnes :

- **Parse fixture** - `crates/pos-fiscal/tests/uc_fixtures.rs` charge le `*_Response.json` publié et le désérialise via les types production `SignResult` / `FdmStatus`.
- **E2e applicatif** - `crates/pos-engine/tests/spf_e2e_basic.rs` pilote la fonction de commande du moteur `do_*` correspondante contre un wiremock FDM amorcé avec cette fixture, puis assert sur l'archive SQLite + les compteurs + l'état de la commande. Ces tests s'exécutent sur la machine de développement ; les commandes `transfer_table` (M122) et `correct_payment` (M132) sont appelées, dans l'application Android, par l'écran des tables et le panier pour la première, par l'onglet Tickets des rapports pour la seconde.

UC	Label événement	Parse fixture	E2e applicatif
M110_signSale	N	✓	✓
M111_signSale_Refund	N	✓	✓
M112A_signSale_RefundPartial / M112A_1_OriginalSale	N	✓	- (pattern non déclaré, couvert via M112B)
M112A_signSale_RefundPartial / M112A_2_PartialRefund	N	✓	- (pattern non déclaré, couvert via M112B)
M112B_signSale_RefundPartial / M112B_1_OriginalSale	N	✓	✓ (chaîné)
M112B_signSale_RefundPartial / M112B_2_PartialRefund	N	✓	✓ (chaîné)
M121_signOrder	P	✓	✓

UC	Label événement	Parse fixture	E2e applicatif
M122_signCostCenterChange	P	✓	✓ (`transfer_table`)
M123_signPrebill	P	✓	✓
M130_signMoneyInOut	F	✓	✓
M131_signDrawerOpen	F	✓	✓
M132_signPaymentCorrection	F	✓	✓ (`correct_payment`)
M133_signMoneyInOut_DrawerDeclaration	F	✓	- (variante de M130)
M140_signWorkIn	S	✓	✓
M141_signWorkOut	S	✓	✓
M150_signInvoice	I	✓	✓ (`m150_invoice` dans `spf_e2e_fixtures.rs` ; `do_sign_invoice_fiscal`, chaîné aux ventes)
M160_signCopy_Sale	C	✓	✓
M161_signCopy_Order	C	✓	✓ (copie d'un événement P)
M170_signWorkIn_Training	T	✓	✓
M171_signSale_Training	T	✓	✓
M171B_signSale_Training	T	✓	- (variante de M171)
M172_signCopy_Training	T	✓	✓ (copie d'un événement T)
Q100_Status	- (statut)	✓	✓ au niveau client (`client_status.rs`, voir ci-dessous)
Q110_Status_multi_lang_message	- (statut)	✓	✓ au niveau client (`client_status.rs`)

Parse fixture : 24 / 24 passent. E2e applicatif : 19 UC rejoués dans `spf_e2e_fixtures.rs` (M122, M132 et M150 compris) + les orchestrations de `spf_e2e_basic.rs` + 4 Reports dans `spf_e2e_reports.rs`. Les lignes "variante" restantes (M133, M171B) sont couvertes transitivement par l'e2e de l'UC parent ; M112A n'est pas un pattern offert par le POS (le remboursement partiel est toujours émis en événement séparé = pattern M112B, déclaré au dossier). M122 (transfert de table) et M132 (correction de paiement) sont implémentés et rejoués (`m122_cost_center_change`, `m132_payment_correction`). M150 (événement I, facture B2B) est implémenté dans le moteur, y compris la build de certification de l'APK 1.0.0, et couvert par le parse fixture et par le rejeu `spf_e2e_fixtures.rs::m150_invoice` (preuves : *RESULTATS_TESTS_UC_FR.md* ; règles et envoi Peppol : *FACTURATION_PEPPOL_FR.md*).

Enveloppe Q100/Q110 - divergence résolue

Les fixtures SPF Q100/Q110 enveloppent le payload sous `data.status` (minuscule), alors que le schéma POS v2.2 documente `Status` (majuscule). `FdmClient::status*` gère désormais les deux orthographes : il interroge `Status` puis retombe sur `status` si le FDM répond avec la forme minuscule. Ce contrat est verrouillé par les 3 tests wiremock de `crates/pos-fiscal/tests/client_status.rs` (argument de langue, fallback d'orthographe, parsing de l'enveloppe multi-langue Q110).

Labels d'événements exercés : les 8 sur 8 - N, P, F, S, I, C, T dans `spf_e2e_basic.rs` et `spf_e2e_fixtures.rs` (I compris, cf. ci-dessus), et R dans `spf_e2e_reports.rs` (M180, M182, M184) en complément du parsing `crates/pos-fiscal/tests/reports_fixtures.rs`.

Dérive de schéma (v1.7, reconduite en v1.8) corrigée dans cette phase

Problème	Fixture qui l'a exposé	Correctif
`SignResult.footer: Vec<String>` était non-optionnel, causant l'échec de désérialisation des réponses M121/M171 parce que le set (v1.7 puis v1.8) omet le champ de ces réponses entièrement.	M121_signOrder, M171_signSale_Training	Ajout de `#[serde(default)]` à `footer` dans `crates/pos-fiscal/src/response.rs`. Le champ manquant désérialise désormais en `vec![]`.

Aucune dérive trouvée côté requête - chaque Variables.json dans le set contient tous les `CommonFields` et les champs spécifiques à l'événement que nos builders `*Input` émettent.

Vérifications de sanité ciblées rédigées à la main

Ce sont des `#[test]`s supplémentaires dans `uc_fixtures.rs` qui épinglent les fixtures les plus tordues sur des valeurs attendues spécifiques, afin que toute dérive future de l'API échoue bruyamment :

- `m110_sign_sale_response_shape` - chemin happy N ; compteurs 15/59 depuis la fixture ; shortSignature, verificationUrl, vatCalc tous présents.
- `m121_sign_order_response_omits_n_only_fields` - prouve que `footer` défaille à `vec![]` quand le wire l'omet.
- `m171_training_sale_has_no_vat_calc` - épinglé selon le changelog (v1.6 du Basic Set).
- `q100_status_response_shape` - épinglé pour garder le désérialiseur de statut FDM honnête.

Reproductibilité

```
# dans le dépôt du moteur, sur la machine de développement
cargo test --package pos-fiscal --test uc_fixtures -- --nocapture
```

Le test roll-up (`basic_set_v1_8_fixtures_deserialize_and_match_schema`) imprime la matrice complète pass/fail sur stderr à chaque exécution ; les quatre tests ciblés épinglent des UC spécifiques par nom.

Hors périmètre ici (reporté aux phases ultérieures)

- Fixtures Reports v1.3 (M180-M184) → Phase 3.
- Fixtures Advanced Set v1.6 (UC200-UC4123) → Phases 5/6/7.
- Sérialisation **requête** octet-équivalente (nous vérifions actuellement `Variables.json` pour l'ensemble de champs requis, pas pour l'égalité octet à octet). La Phase 4 introduit la sérialisation JSON canonique, après quoi nous ajouterons un test d'équivalence de requête pour le dossier cert.